

3. Multimodal Visual Understanding + Depth Camera Distance Question Answering

3. Multimodal Visual Understanding + Depth Camera Distance Question Answering

Introduction

Operation Steps

Preparatory Steps

Getting Started

Usage Effect

Code Summary

Supplement: Radar Ranging

Introduction

In this tutorial, we will learn how to issue commands via chat with Muto in the Dify interface.

Operation Steps

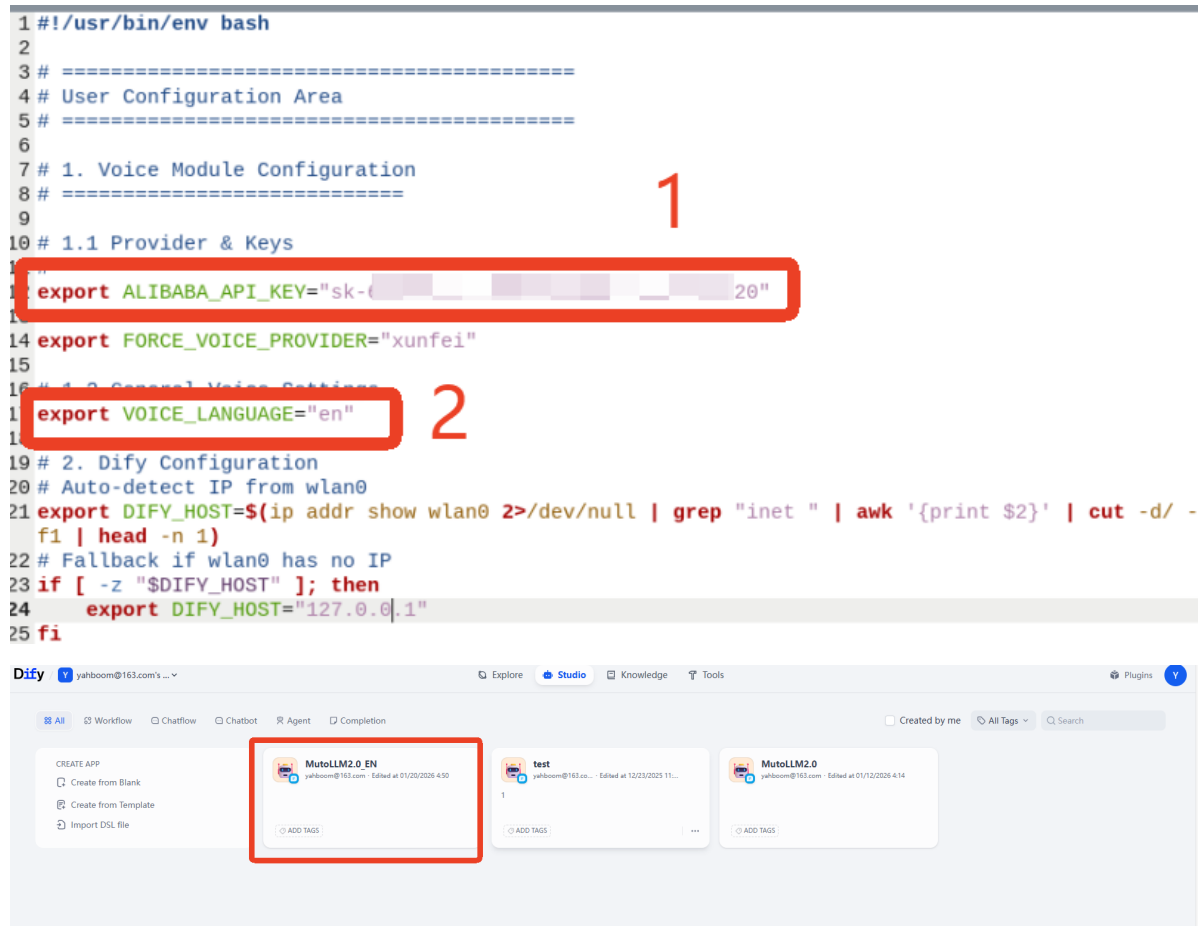
Preparatory Steps

This lesson requires that the Dify terminal be launched beforehand, and all configuration items have been configured as in the previous section.

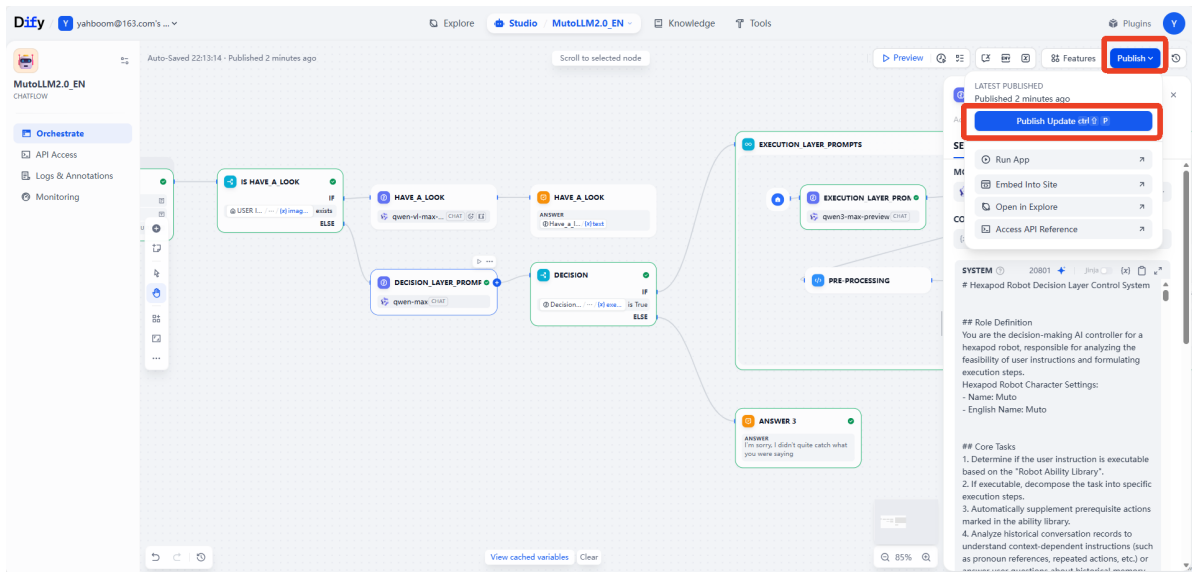
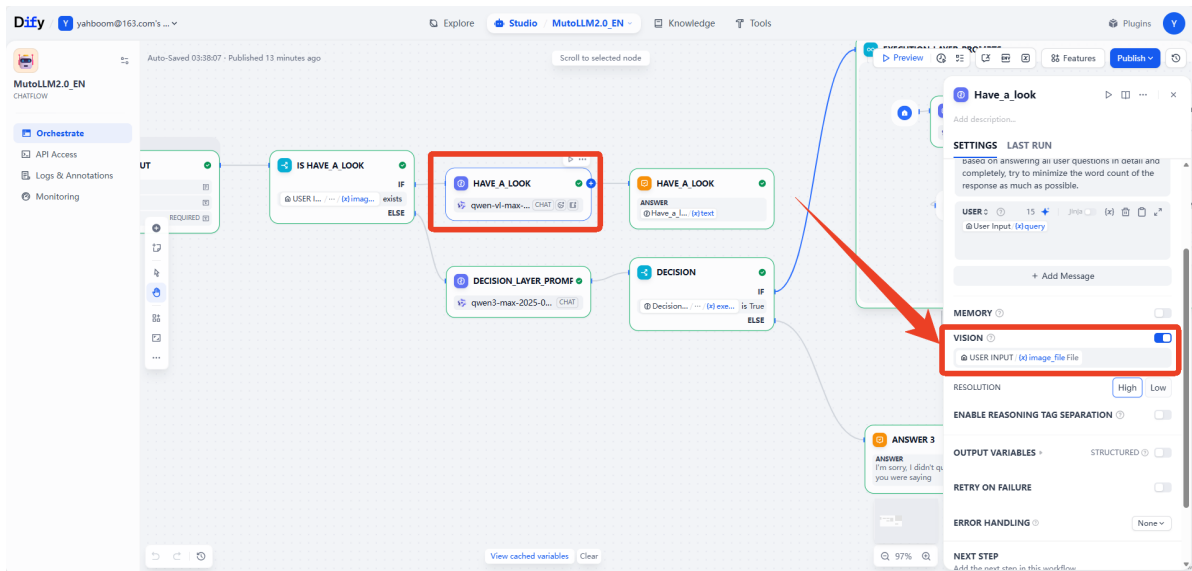
```
1 #!/usr/bin/env bash
2
3 # =====
4 # User Configuration Area
5 # =====
6
7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 export ALIBABA_API_KEY="sk-20"
12
13 export FORCE_VOICE_PROVIDER="xunfei"
14
15 # 1.2 General Voice Settings
16 export VOICE_LANGUAGE="en"
17
18 # 2. Dify Configuration
19 # Auto-detect IP from wlan0
20 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
21 # Fallback if wlan0 has no IP
22 if [ -z "$DIFY_HOST" ]; then
23   export DIFY_HOST="127.0.0.1"
24 fi
```

1

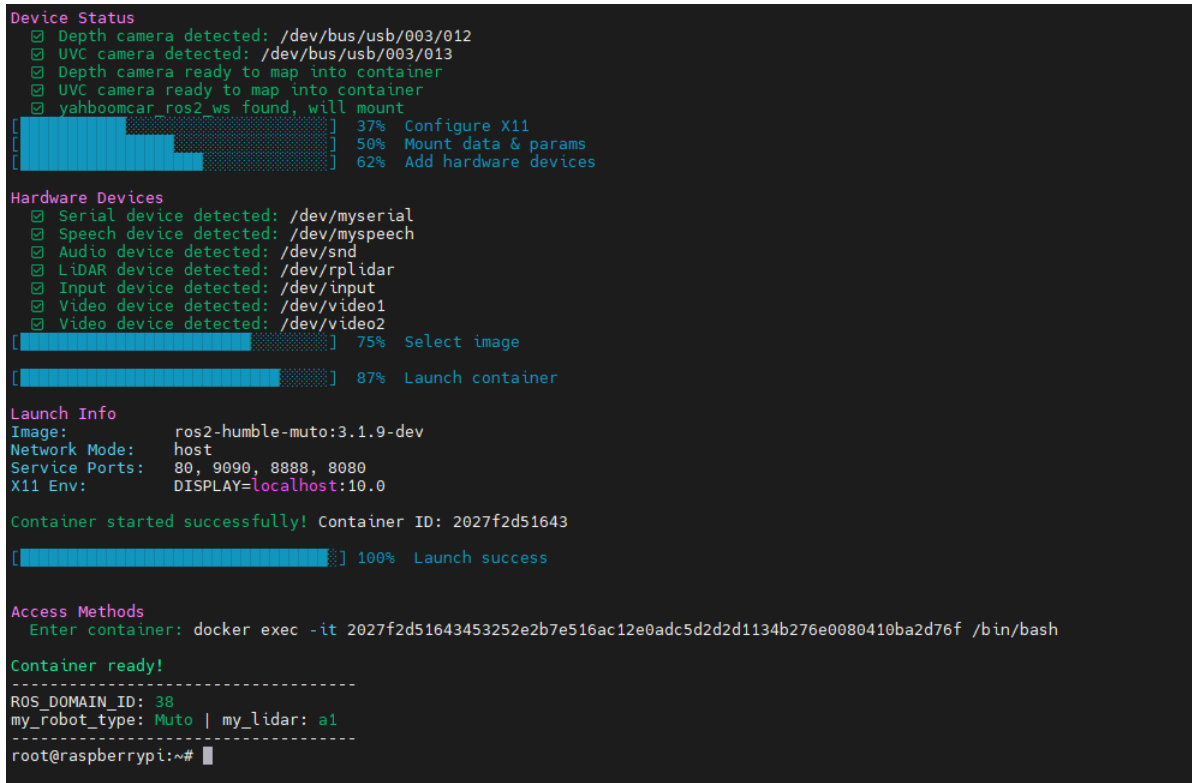
2



The screenshot shows the Dify interface with a terminal window at the top displaying the configuration script. The script sets the Alibaba API key to "sk-20" and the voice language to "en". The Dify interface below shows the configuration of the MutoLLM2.0 EN model, with the "test" button highlighted.



This lesson requires MutoRS to be powered on. Following the steps in the previous chapters, start the large model terminal in Docker.

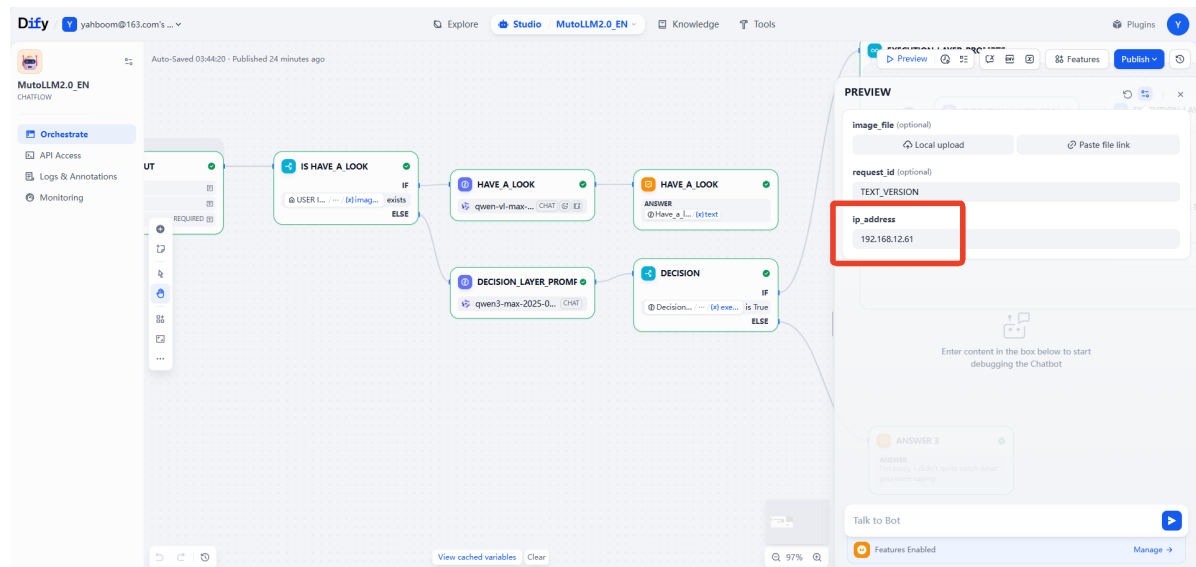


```
cd muto-11m-2.0/ && ./rebuild_and_launch.sh --no-voice
```

```
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239218] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

Getting Started

For the large text model, you need to enter Muto's IP address here, which is the address used to access the Dify page.

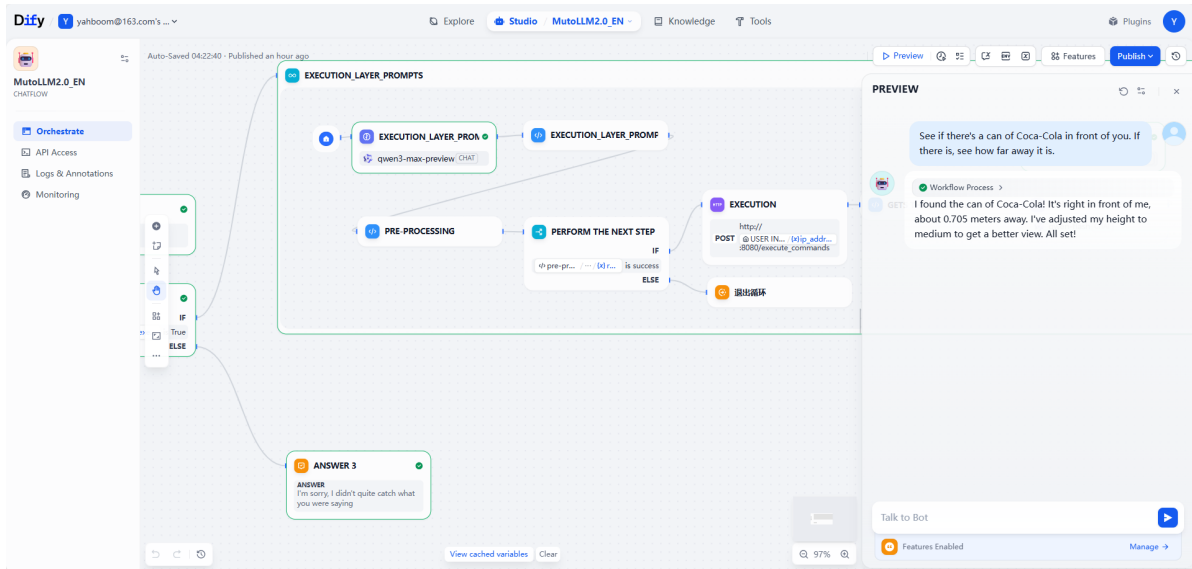


In this tutorial section, we will integrate the large model into our depth camera.

Usage Effect

The command used in this section is: "See if there's a can of Coca-Cola in front of you. If there is, see how far away it is."

The effect is as follows:



You can change the object to query according to the actual scenario.

Code Summary

The key function used in this section is:

`get_depth_at_point(x, y)`: Gets the distance (in meters) at the specified coordinates (x, y) in the depth camera image. It needs to be used in conjunction with the results of visual recognition.

This function is defined in the path

`\packages\muto_hexapod_lib\muto_hexapod_lib\Largemodel\sensor_manager.py`

The core code is as follows:

```
def get_depth_at_point(self, x: int, y: int, timeout_s: float = 5.0) ->
tuple:
    """
    获取指定坐标点的深度信息（智能节点管理）
    Get depth information at specified coordinates (with intelligent node
    management)

    Args:
        x: 图像中的x坐标 X coordinate in the image
        y: 图像中的y坐标 Y coordinate in the image
        timeout_s: 超时时间(秒) Timeout in seconds

    Returns:
        tuple: (success: bool, message: str, data: dict)
            success: 执行是否成功 whether execution was successful
            message: 执行结果消息 Execution result message
            data: 深度信息 Depth information
    """
    depth_topic = "/camera/depth/image_raw"

    try:
        # Intelligent Node Management: Check Camera Initial Status
        camera_was_running = False
        camera_started_by_us = False

        camera_success, camera_msg, camera_data = self.get_camera_status()
```

```

        camera_was_running = camera_success and camera_data.get('status') ==
'running'

        # If the camera is not running, start the camera first.
        if not camera_was_running:
            start_success, start_msg, start_data =
self.start_camera(wait_time_s=5.0)
            if not start_success:
                return False, f"无法启动相机 Failed to start camera:
{start_msg}", {
                    "x": x,
                    "y": y,
                    "camera_start_attempted": True,
                    "camera_start_success": False,
                    "error": "camera_start_failed"
                }
            camera_started_by_us = True

# Execute core depth acquisition function
# 开始获取坐标点的深度信息 Starting to get depth info at point
# 深度话题 Depth topic

# Check necessary dependencies
try:
    import rclpy
    from rclpy.node import Node
    from sensor_msgs.msg import Image
    from cv_bridge import CvBridge
    import cv2
    import numpy as np
    # 所有必要依赖可用 All required dependencies available
except ImportError as import_error:
    # 缺少必要依赖 Missing required dependencies
    # 请安装: pip install opencv-python
    # Please install: pip install opencv-python
    return False, f"缺少必要依赖 Missing required dependencies:
{str(import_error)}", {"x": x, "y": y, "error": "missing_dependencies"}

# Create a depth image capture node
import uuid
depth_node_id = str(uuid.uuid4())[:8] # Use the first 8 digits of
the UUID as the unique identifier

class DepthCaptureNode(Node):
    def __init__(self):
        super().__init__(f'depth_capture_node_{depth_node_id}')
        self.bridge = CvBridge()
        self.depth_received = False
        self.depth_image = None
        self.subscription = self.create_subscription(
            Image,
            depth_topic,
            self.depth_callback,
            10
        )
        # 已订阅深度话题 Subscribed to depth topic

    def depth_callback(self, msg):

```

```

        try:
            # Convert ROS depth image messages to OpenCV images
            # Depth images are typically 16-bit single-channel
            depth_image = self.bridge.imgmsg_to_cv2(msg,
desired_encoding="passthrough")
            self.depth_image = depth_image
            self.depth_received = True
            # 成功接收深度图像 Successfully received depth image
        except Exception as e:
            print(f"深度图像转换失败 Depth image conversion failed:
{str(e)}")

            # 深度图像转换失败 Depth image conversion failed

        # Initialize ROS2 (if it has not been initialized yet)
        if not rclpy.ok():
            rclpy.init()

        # Create nodes and wait for depth image
        node = DepthCaptureNode()

        # 等待深度图像数据 waiting for depth image data

        # waiting for depth image data - Add error handling and retry
mechanism
        start_time = time.time()
        spin_retry_count = 0
        max_spin_retries = 3

        while not node.depth_received and (time.time() - start_time) <
timeout_s:
            try:
                rclpy.spin_once(node, timeout_sec=0.1)
                spin_retry_count = 0 # Reset retry count
            except Exception as spin_error:
                spin_retry_count += 1
                error_msg = str(spin_error)

                # Check if it's an "event index too big" error.
                if "event index too big" in error_msg or "index" in
error_msg.lower():
                    if spin_retry_count <= max_spin_retries:
                        print(f"⚠ ROS2事件循环错误 (深度获取重试
{spin_retry_count}/{max_spin_retries}): {error_msg}")
                        time.sleep(0.2) # Retry after a short delay
                        continue
                    else:
                        print(f"✖ ROS2事件循环错误超过最大重试次数 (深度获取):
{error_msg}")
                        node.destroy_node()
                        return False, f"ROS2事件循环错误 (深度获取):
{error_msg}", {"x": x, "y": y, "error": "ros2_event_loop_error_depth"}
                    else:
                        # Other types of errors
                        print(f"✖ ROS2 spin_once错误 (深度获取): {error_msg}")
                        node.destroy_node()
                        return False, f"ROS2 spin_once错误 (深度获取):
{error_msg}", {"x": x, "y": y, "error": "ros2_spin_error_depth"}

```

```

if node.depth_received and node.depth_image is not None:
    # Check if the coordinates are within the image range
    height, width = node.depth_image.shape[:2]

    if x < 0 or x >= width or y < 0 or y >= height:
        # 坐标超出图像范围 Coordinates out of image bounds
        # 图像尺寸 Image size
        # 请求坐标 Requested coordinates
        node.destroy_node()
        return False, f"坐标超出图像范围 Coordinates out of bounds:
({x}, {y}), image size: {width}x{height}", {
            "x": x, "y": y,
            "image_width": width,
            "image_height": height,
            "error": "coordinates_out_of_bounds"
        }

    # Get the depth value of a specified coordinate point
    depth_value = node.depth_image[y, x]

    # Depth values are usually expressed in millimeters, which can be
    converted to meters.
    if node.depth_image.dtype == np.uint16:
        # 16-bit depth image, typically in millimeters
        depth_meters = float(depth_value) / 1000.0
        depth_mm = float(depth_value)
    elif node.depth_image.dtype == np.float32:
        # 32-bit floating-point depth image, usually in meters
        depth_meters = float(depth_value)
        depth_mm = float(depth_value) * 1000.0
    else:
        # Other formats, assuming millimeters are the unit
        depth_meters = float(depth_value) / 1000.0
        depth_mm = float(depth_value)

    # Check if the depth value is valid (0 usually indicates an
    invalid depth).
    is_valid = bool(depth_value > 0) # Convert to Python's native
    bool type

    # 深度信息获取成功 Depth information retrieved successfully
    # 坐标 Coordinates
    # 原始深度值 Raw depth value
    # 深度(米) Depth (meters)
    # 深度(毫米) Depth (millimeters)
    # 数据类型 Data type
    # 有效性 Valid

    # Obtain depth information of the surrounding area (3x3 region)
    surrounding_depths = []
    for dy in range(-1, 2):
        for dx in range(-1, 2):
            nx, ny = x + dx, y + dy
            if 0 <= nx < width and 0 <= ny < height:
                surrounding_depth = node.depth_image[ny, nx]
                if node.depth_image.dtype == np.uint16:

```



```

surrounding_depth_m = float(surrounding_depth) /
1000.0

elif node.depth_image.dtype == np.float32:
    surrounding_depth_m = float(surrounding_depth)
else:
    surrounding_depth_m = float(surrounding_depth) /
1000.0

surrounding_depths.append({
    "x": nx, "y": ny,
    "depth_raw": float(surrounding_depth),
    "depth_meters": surrounding_depth_m,
    "valid": bool(surrounding_depth > 0) # Convert
to Python's native bool type
})

# Clean up nodes
node.destroy_node()

# Intelligent Node Management: If it's a camera we started, close
it after execution.
if camera_started_by_us:
    try:
        self.stop_camera(wait_time_s=1.0)
    except Exception as stop_error:
        print(f"⚠ 自动关闭相机警告: {stop_error}")

    return True, "深度信息获取成功 Depth information retrieved
successfully", {
    "x": x,
    "y": y,
    "depth_raw": float(depth_value),
    "depth_meters": depth_meters,
    "depth_millimeters": depth_mm,
    "is_valid": is_valid,
    "image_size": {"width": width, "height": height},
    "data_type": str(node.depth_image.dtype),
    "surrounding_area": surrounding_depths,
    "method": "cv_bridge_depth",
    "camera_auto_managed": camera_started_by_us
}
else:
    # 深度图像获取超时 Depth image capture timeout
    # 请检查深度相机是否正常发布数据 Please check if depth camera is
publishing data normally
    node.destroy_node()

    # Intelligent Node Management: If it's a camera we started, close
it after execution.
    if camera_started_by_us:
        try:
            self.stop_camera(wait_time_s=1.0)
        except Exception as stop_error:
            print(f"⚠ 自动关闭相机警告: {stop_error}")

        return False, f"深度图像获取超时 Depth image capture timeout
({timeout_s}s)", {
            "x": x, "y": y,
            "timeout": timeout_s,

```

```

        "suggestion": "check_depth_camera_publishing",
        "camera_auto_managed": camera_started_by_us
    }

except Exception as e:
    # 深度信息获取过程中发生错误 Error during depth information retrieval
    # Intelligent Node Management: If it's a camera we started, close it
    after execution.
    if camera_started_by_us:
        try:
            self.stop_camera(wait_time_s=1.0)
        except Exception as stop_error:
            print(f"⚠️ 自动关闭相机警告: {stop_error}")

    return False, f"深度信息获取过程中发生错误 Error during depth information
retrieval: {str(e)}", {
        "x": x, "y": y,
        "error": str(e),
        "camera_auto_managed": camera_started_by_us
    }
}

```

Supplement: Radar Ranging

Besides using a depth camera, the Muto can also perform 360-degree ranging using its built-in LiDAR. Before using this function, the LiDAR needs to be configured (see 0. MutoRS Dedicated Large Model Configuration and Related Content Instructions).

The prompt is as follows:

"Please scan the environment with the LiDAR to see how far away the obstacle is directly in front of you."

The screenshot displays the Dify AI platform interface for a workflow named "MutoLLM2.0_EN". The workflow is titled "Auto-Saved 05:49:55 - Published 2 hours ago". The main canvas shows a sequence of steps: "EXECUTION_LAYER_PROMPT" (with a prompt "qwen3-max-preview: CHAT"), "PRE-PROCESSING", "PERFORM THE NEXT STEP" (with a conditional logic "if pre-pr... / ... (X) ... is success"), "EXECUTION" (with a POST request to "http://@USER IN... (http_addr.../8080/execute_commands"), "GETS THE ORIGINAL DECIS", and "COMPRESSION".

The "PREVIEW" panel on the right shows the prompt: "Please use LiDAR to scan the environment and see how far away the obstacle is directly in front of you." Below the prompt, it displays the "Workflow Process" and the "The result of this execution is as follows":

- (Start LiDAR) : (success: true, message: "Lidar node started successfully: a1; Height adjustment successful: Medium"; executed_commands: ["robot speak: Starting LiDAR scanning now.", "start lidar: success", "adjust height: level 2, Medium"]); execution_time: 13.53)
- (Original Action Requirement) : Get LiDAR distance at specified angle (directly ahead) (Compressed Execution Result, Data) : ("success": true, "data": {"target_angle": 0, "actual_angle": -1.002e-07, "distance": 1.022, "is_within_tolerance": true}, "message": "Successfully got lidar data at angle -0.0", distance: 1.022m")
- (Original Action Requirement) : Broadcast the measured distance to the obstacle directly in front (Compressed Execution Result, Data)

The interface also includes a "Talk to Bot" section at the bottom right with a "Features Enabled" button and a "Manage" link.

```
[muto_controller_node-1] Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] Serial data receiving started
[muto_controller_node-1] Voice wake detector started successfully
[muto_controller_node-1] Wake detection started
[muto_controller_node-1] Voice assistant auto-start: True
[muto_controller_node-1] ROS2 publisher created successfully
[muto_controller_node-1] Status timer and health check created successfully
[muto_controller_node-1] Muto Controller Node initialized successfully
[muto_controller_node-1] Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [1094]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
[muto_controller_node-1] Mode: Sync (Blocking)
[muto_controller_node-1] Plan Data: ["robot_speak(text='正在启动激光雷达')", "start_lidar(wait_time_s=10.0, check_before_start=True)"]
[muto_controller_node-1] [Muto reply] 正在启动激光雷达
[muto_controller_node-1] Trying to start camera ...
[muto_controller_node-1] {'event': 'execute_commands', "success": true, "executed_count": 3, "failed_command_id": null, "duration_ms": 13580}
[muto_controller_node-1] Mode: Sync (Blocking)
[muto_controller_node-1] Plan Data: ["robot_speak(text='正在测量正前方距离')", "get_lidar_range_at_angle(target_angle=0.0)"]
[muto_controller_node-1] [Muto reply] 正在测量正前方距离
[muto_controller_node-1] 360度雷达数据调试信息 Debug info for 360-degree lidar data:
[muto_controller_node-1] 总点数 Total points: 2020, 有效点数 Valid points: 1864
[muto_controller_node-1] 角度范围 Angle range: -3.142 ~ 3.142 弧度 radians
[muto_controller_node-1] 角度范围 Angle range: -180.0° ~ 180.0°
[muto_controller_node-1] 角度增量 Angle increment: 0.003112 弧度 radians
[muto_controller_node-1] 角度增量 Angle increment: 0.178°
[muto_controller_node-1] 前10个角度 First 10 angles (radians): ['-3.142', '-3.138', '-3.135', '-3.132', '-3.129', '-3.126', '-3.123', '-3.120', '-3.117', '-3.114']
[muto_controller_node-1] 前10个角度 First 10 angles (degrees): ['-180.0°', '-179.8°', '-179.6°', '-179.5°', '-179.3°', '-179.1°', '-178.9°', '-178.6°', '-178.4°', '-178.2°']
[muto_controller_node-1] 后10个角度 Last 10 angles (radians): ['3.114', '3.117', '3.120', '3.123', '3.126', '3.129', '3.132', '3.135', '3.138', '3.142']
[muto_controller_node-1] 后10个角度 Last 10 angles (degrees): ['178.4°', '178.6°', '178.8°', '178.9°', '179.1°', '179.3°', '179.5°', '179.6°', '179.8°', '180.0°']
[muto_controller_node-1] 角度匹配调试信息 Angle matching debug info:
[muto_controller_node-1] 目标角度 Target angle: 0.000 弧度 (0.0°)
[muto_controller_node-1] 角度容差 Angle tolerance: 0.200 弧度 (11.5°)
[muto_controller_node-1] 数据点总数 Total data points: 2020
[muto_controller_node-1] 实际角度范围 Actual angle range: -3.142 ~ 3.142 弧度 (-180.0° ~ 180.0°)
[muto_controller_node-1] 目标角度是否在数据范围内 Target angle in data range: True
[muto_controller_node-1] 最接近的5个角度点 Closest 5 angle points:
[muto_controller_node-1] [1] 索引1010: 0.002弧度 (0.1°), 差异: 0.002弧度 (0.1°)
[muto_controller_node-1] [2] 索引1009: -0.002弧度 (-0.1°), 差异: 0.002弧度 (0.1°)
[muto_controller_node-1] [3] 索引1011: 0.005弧度 (0.3°), 差异: 0.005弧度 (0.3°)
[muto_controller_node-1] [4] 索引1008: -0.005弧度 (-0.3°), 差异: 0.005弧度 (0.3°)
[muto_controller_node-1] [5] 索引1012: 0.008弧度 (0.4°), 差异: 0.008弧度 (0.4°)
[muto_controller_node-1] {'event': 'execute_commands', "success": true, "executed_count": 3, "failed_command_id": null, "duration_ms": 6559}
[muto_controller_node-1] Mode: Async (Background)
[muto_controller_node-1] Plan Data: ["robot_speak(text='正前方障碍物距离约2米。')"]
[muto_controller_node-1] [Muto reply] 正前方障碍物距离约2米。
```

The core code is as follows:

```
def get_lidar_data(self, timeout_s: float = 5.0, sample_count: int = 1) -> tuple:
    """
    读取雷达节点数据（智能节点管理：自动检查并启动雷达节点）
    Read lidar node data (Smart node management: automatically check and start lidar node)

    Args:
        timeout_s: 读取超时时间(秒) Read timeout in seconds
        sample_count: 采样次数 Number of samples to collect

    Returns:
        tuple: (success: bool, message: str, data: dict)
            success: 读取是否成功 whether read was successful
            message: 读取结果消息 Read result message
            data: 雷达数据信息 Lidar data information

    Examples:
        success, msg, data = sensor_manager.get_lidar_data()
        """
    # Intelligent Node Management: Check radar status and activate when needed.
    lidar_started_by_us = False
    lidar_status_success, _, lidar_status_data = self.check_lidar_status()

    if not lidar_status_success or not lidar_status_data.get('node_running', False):
        # Radar is not running, try starting it.
        start_success, start_msg, _ = self.start_lidar(wait_time_s=3.0)
        if not start_success:
            return False, f"无法启动雷达节点 Failed to start lidar node: {start_msg}", {
                "lidar_auto_managed": False,
                "start_error": start_msg
            }
    }
```

```

        lidar_started_by_us = True

    try:
        lidar_type = self._get_lidar_type()
        topic_name = "/scan"

        # Check if the topic is active
        topic_success, topic_msg, topic_data =
self.check_ros2_topic_active(topic_name, "sensor_msgs/msg/LaserScan")
        if not topic_success or not topic_data.get('active', False):
            # Intelligent Node Management: If it's a radar system we
initiated, shut it down after execution.
            if lidar_started_by_us:
                try:
                    self.stop_lidar(wait_time_s=1.0)
                except Exception as stop_error:
                    print(f"⚠ 自动关闭雷达警告: {stop_error}")

            return False, f"雷达话题未活跃 Lidar topic not active:
{topic_name}", {
                "lidar_type": lidar_type,
                "topic_name": topic_name,
                "topic_active": False,
                "error": topic_msg,
                "lidar_auto_managed": lidar_started_by_us
            }

        # Read Radar Data - Compatible with ROS2 Humble Version, Optimized
Blocking Time
        # Use the `--once` parameter to obtain a complete frame of data,
avoiding incomplete data due to manual truncation.
        # Add the `--full-length` parameter to ensure that array data is not
omitted.
        cmd = ["ros2", "topic", "echo", "--once", "--full-length",
topic_name]

        try:
            # Start the process using Popen
            process = subprocess.Popen(
                cmd,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                text=True
            )

            # wait for data output, using the full timeout period.
            output_lines = []
            try:
                stdout, stderr = process.communicate(timeout=timeout_s)
                if stdout:
                    output_lines = stdout.splitlines()
            except subprocess.TimeoutExpired:
                process.kill()
            # Intelligent Node Management: If it's a radar system we
initiated, shut it down after execution.
            if lidar_started_by_us:
                try:
                    self.stop_lidar(wait_time_s=1.0)

```

```

        except Exception as stop_error:
            print(f"⚠️ 自动关闭雷达警告: {stop_error}")
            return False, f"读取雷达数据超时 Lidar data read timeout:
{timeout_s}s", {
                "lidar_type": lidar_type,
                "topic_name": topic_name,
                "data_received": False,
                "error": "timeout",
                "lidar_auto_managed": lidar_started_by_us
            }

    # Process the acquired data
    if output_lines and len(output_lines) > 0:
        # Analyze basic information
        scan_info = {
            "lidar_type": lidar_type,
            "topic_name": topic_name,
            "data_received": True,
            "raw_data_length": len('\n'.join(output_lines)),
            "timestamp": time.time(),
            "lines_received": len(output_lines)
        }

        # Parse the various fields of the LaserScan message
        ranges_data = []
        intensities_data = []
        in_ranges_section = False
        in_intensities_section = False

        for line in output_lines:
            line = line.strip()
            try:
                # Parse header information
                if "frame_id:" in line:
                    scan_info["frame_id"] = line.split(":")
[1].strip()

                # Analysis Angle Information
                elif "angle_min:" in line:
                    scan_info["angle_min"] = float(line.split(":")
[1].strip())

                elif "angle_max:" in line:
                    scan_info["angle_max"] = float(line.split(":")
[1].strip())

                elif "angle_increment:" in line:
                    scan_info["angle_increment"] =
float(line.split(":")[1].strip())

                # Parse time information
                elif "time_increment:" in line:
                    scan_info["time_increment"] =
float(line.split(":")[1].strip())

                elif "scan_time:" in line:
                    scan_info["scan_time"] = float(line.split(":")
[1].strip())

                # Analyze distance range
                elif "range_min:" in line:
                    scan_info["range_min"] = float(line.split(":")
[1].strip())

                elif "range_max:" in line:

```

```

scan_info["range_max"] = float(line.split(":")
[1].strip())

# Start checking the ranges array
elif line == "ranges:":
    in_ranges_section = True
    in_intensities_section = False
# Start checking the intensities array
elif line == "intensities:":
    in_intensities_section = True
    in_ranges_section = False
# Parse ranges data
elif in_ranges_section and line.startswith("- "):
    try:
        value_str = line[2:].strip() # Remove the
        "-" prefix

        if value_str == ".inf":
            # Convert infinity values to maximum
            measurement range values to avoid JSON serialization errors.

        ranges_data.append(scan_info.get("range_max", 30.0))
        elif value_str == "-.inf":
            # Convert negative infinity to the
            minimum measurement range value

        ranges_data.append(scan_info.get("range_min", 0.0))
        elif value_str == "nan":
            # Convert NaN values to 0 to indicate
            invalid measurements.

            ranges_data.append(0.0)
        else:
            value = float(value_str)
            # Check if it is a valid floating-point
            number

            if value != value or value ==
            float('inf') or value == float('-inf'):
                ranges_data.append(0.0) # Convert
                invalid values to 0

            else:
                ranges_data.append(value)
        except ValueError:
            continue
# Parse intensities data
elif in_intensities_section and line.startswith("-
"):
    try:
        value_str = line[2:].strip() # Remove the
        "-" prefix

        if value_str == ".inf" or value_str ==
        "-.inf" or value_str == "nan":
            # Convert invalid strength values to 0
            to avoid JSON serialization errors.

            intensities_data.append(0.0)
        else:
            value = float(value_str)
            # Check if it is a valid floating-point
            number

            if value != value or value ==
            float('inf') or value == float('-inf'):

```

```

intensities_data.append(0.0) #
Convert invalid values to 0
        else:
            intensities_data.append(value)
        except ValueError:
            continue
        # Stop parsing the current array when other fields are
detected
        elif line.endswith(":") and not line.startswith("-
"):
            in_ranges_section = False
            in_intensities_section = False
        except (ValueError, IndexError):
            continue # Ignore lines with parsing errors

# Add parsed array data
if ranges_data:
    scan_info["ranges"] = ranges_data
    scan_info["ranges_count"] = len(ranges_data)
    # Calculate the statistics of valid distance data
(excluding 0 values, as 0 indicates an invalid measurement).
    valid_ranges = [r for r in ranges_data if r > 0.0 and r
!= float('inf') and r != float('-inf') and r == r] # Exclude inf, nan and 0
values
    if valid_ranges:
        scan_info["valid_ranges_count"] = len(valid_ranges)
        scan_info["min_range"] = min(valid_ranges)
        scan_info["max_range"] = max(valid_ranges)
        scan_info["avg_range"] = sum(valid_ranges) /
len(valid_ranges)
    else:
        scan_info["valid_ranges_count"] = 0

if intensities_data:
    scan_info["intensities"] = intensities_data
    scan_info["intensities_count"] = len(intensities_data)

# Generate sample_points (angle-distance pairs)
if "angle_min" in scan_info and "angle_increment" in
scan_info and ranges_data:
    sample_points = []
    angle_min = scan_info["angle_min"]
    angle_increment = scan_info["angle_increment"]

    for i, range_val in enumerate(ranges_data):
        angle = angle_min + i * angle_increment
        sample_points.append({
            "angle": angle,
            "distance": range_val,
            "intensity": intensities_data[i] if i <
len(intensities_data) else 0.0
        })

    scan_info["sample_points"] = sample_points
else:
    # If critical data is missing, create empty
sample_points to maintain compatibility.
    scan_info["sample_points"] = []

```

```

        scan_info["data_incomplete"] = True
        scan_info["missing_fields"] = []
        if "angle_min" not in scan_info:
            scan_info["missing_fields"].append("angle_min")
        if "angle_increment" not in scan_info:

scan_info["missing_fields"].append("angle_increment")
            if not ranges_data:
                scan_info["missing_fields"].append("ranges_data")

        # Intelligent Node Management: If it's a radar system we
initiated, shut it down after execution.
        if lidar_started_by_us:
            try:
                self.stop_lidar(wait_time_s=1.0)
            except Exception as stop_error:
                print(f"⚠ 自动关闭雷达警告: {stop_error}")

        # Add automatic management of status to returned data
        scan_info["lidar_auto_managed"] = lidar_started_by_us

        return True, f"成功读取雷达数据 Successfully read lidar data:
{lidar_type}", scan_info
    else:
        # Intelligent Node Management: If it's a radar system we
initiated, shut it down after execution.
        if lidar_started_by_us:
            try:
                self.stop_lidar(wait_time_s=1.0)
            except Exception as stop_error:
                print(f"⚠ 自动关闭雷达警告: {stop_error}")

        return False, f"雷达数据为空 Empty lidar data: {topic_name}", {
            "lidar_type": lidar_type,
            "topic_name": topic_name,
            "data_received": False,
            "error": "empty_data",
            "lidar_auto_managed": lidar_started_by_us
        }

    except Exception as e:
        # Intelligent Node Management: If it's a radar system we
initiated, shut it down after execution.
        if lidar_started_by_us:
            try:
                self.stop_lidar(wait_time_s=1.0)
            except Exception as stop_error:
                print(f"⚠ 自动关闭雷达警告: {stop_error}")

        return False, f"读取雷达数据出错 Error reading lidar data:
{str(e)}", {
            "lidar_type": lidar_type,
            "topic_name": topic_name,
            "data_received": False,
            "error": str(e),
            "lidar_auto_managed": lidar_started_by_us
        }

```



```

except subprocess.TimeoutExpired:
    # Intelligent Node Management: If it's a radar system we initiated,
    shut it down after execution.
    if lidar_started_by_us:
        try:
            self.stop_lidar(wait_time_s=1.0)
        except Exception as stop_error:
            print(f"⚠ 自动关闭雷达警告: {stop_error}")

    return False, f"读取雷达数据超时 Lidar data read timeout:
{timeout_s}s", {
        "lidar_type": lidar_type,
        "topic_name": topic_name,
        "data_received": False,
        "error": "timeout",
        "lidar_auto_managed": lidar_started_by_us
    }
except Exception as e:
    # Intelligent Node Management: If it's a radar system we initiated,
    shut it down after execution.
    if lidar_started_by_us:
        try:
            self.stop_lidar(wait_time_s=1.0)
        except Exception as stop_error:
            print(f"⚠ 自动关闭雷达警告: {stop_error}")

    return False, f"读取雷达数据出错 Error reading lidar data: {str(e)}", {
        "lidar_type": lidar_type,
        "error": str(e),
        "lidar_auto_managed": lidar_started_by_us
    }
}

```